

Dockerized Deployments: Dev Container to Production, Fully Automated

Take a containerized app from "I'm editing code in a VS Code Dev Container" to "running on my VPS, automatically, on every push to main." Companion to [vps-from-zero](#), which provisions the server this guide deploys to.

NOTE. LAST VALIDATED: 2026-05. Tool versions reflect current stable: Python 3.12, Postgres 16, [uv](#) current, Docker Compose v2 (with [develop.watch.sync](#)), GHA action majors [actions/checkout@v4](#) / [docker/setup-buildx-action@v3](#) / [docker/build-push-action@v5](#) / [docker/metadata-action@v5](#) / [appleboy/ssh-action@v1](#). Bump and re-validate annually per CLAUDE.md standard #5.

This document is structured around **the lifecycle of one code change** — the journey from your fingers on a keyboard to a container running on the VPS. Every section explains both what you're doing and why — with enough mechanical detail that when something breaks, you can find the layer it broke at.

What you'll have at the end

A pipeline where every `git push origin main`:

1. GitHub Actions checks out your code.
2. Buildx builds a Docker image of your runtime stage.
3. The image is tagged with `:latest` and `:sha-<commit>` and pushed to **GHCR** (GitHub's container registry).
4. A second workflow SSHes into your VPS as the `deploy` user.
5. Compose runs any pending database migrations.
6. Compose pulls the new image and restarts the relevant container.
7. The new code is live, end-to-end in roughly 90 seconds.

You'll also have a **VS Code Dev Container** for local development that is literally the same Docker image (well, the dev stage of it) — so "works on my machine" can no longer differ from "works in production."

Prerequisites

- A working VPS with Docker, the `deploy` user, the shared network, and the shared Postgres — set up per [vps-from-zero](#). Stop here and do that first if you haven't.
 - A GitHub account.
 - VS Code with the **Remote - WSL** and **Dev Containers** extensions installed.
 - Docker Desktop on Windows with WSL2 integration enabled, OR [docker.io](#) installed in your WSL distro. The container engine has to actually work for any of this.
-

Table of contents

1. [The shape: how a code change reaches production](#)
 2. [The five principles](#)
 3. [The Dockerfile: the artifact that ties dev to prod](#)
 4. [The repo, end to end](#)
 5. [Local development with VS Code Dev Containers](#)
 6. [Day-to-day development cycles](#)
 7. [The registry: GHCR](#)
 8. [CI: building images with GitHub Actions](#)
 9. [CD: deploying to the VPS](#)
 10. [Database migrations in the pipeline](#)
 11. [End-to-end: the full lifecycle of one code change](#)
 12. [Rollbacks](#)
 13. [Adding a second \(or fifth\) app](#)
 14. [Reverse proxy with Caddy \(HTTPS for web apps\)](#)
 15. [Troubleshooting](#)
 16. [Alternatives considered](#)
 17. [Quick reference](#)
-

1. The shape: how a code change reaches production

[Diagram: see the online version on GitHub for the interactive flowchart.]

Internalize this diagram. Everything in the rest of the document is about making one of these arrows reliable.

Where the journey starts: you, in VS Code, with your hands on a keyboard. The editor is connected to a running Docker container — the **Dev Container** — that has the exact same Python (or Node, etc.) version, the exact same OS packages, and the exact same dependencies as the container that will run in production. Your code is in `src/` on your laptop's filesystem, sync-mounted into `/app/src` inside the container.

Where the journey ends: a container running on the VPS, pulled from GHCR, listening on the shared Docker network, and connected to the shared Postgres. The same image. The same code. The "automation" is the four arrows between those two points: `git push` → `build.yml` → push to GHCR → `deploy.yml` → SSH to VPS → `docker compose up -d`. If any of those break, you find which arrow failed and debug that one layer.

2. The five principles

These show up over and over. Get them in your bones and the rest is just configuration.

2.1. One image, many environments

The Docker image you push to GHCR is the same artifact that runs in your local Dev Container and on the VPS. The only thing that varies between environments is **configuration injected at runtime** — through environment variables. Same binary, different inputs.

This is the property that eliminates "works on my machine." If a bug exists in prod, it can be reproduced locally by running the same image with the same env vars.

Practical consequence: never bake env-specific stuff into the image. Never `COPY .env` inside a Dockerfile. The image is built once and runs many times in many environments; the env is the differentiator.

2.2. Stacks, not monoliths

Each app is its own `docker-compose` "stack" — its own directory on the VPS, its own compose file, its own `.env`. Shared services (Postgres, Caddy) live in the separate `infra/` stack. Apps talk to shared services over a **manually-created Docker network** that exists outside any compose project.

This means redeploying app A never touches app B or the database. The blast radius of a deploy is bounded to one stack.

Practical consequence: the directory layout matters. `~/infra/`, `~/app1/`, `~/app2/`. Each with its own compose file. No "one big compose file for everything" temptation, even when you only have one app — because once you have a second app, untangling them is painful.

2.3. Secrets never go inside images

Docker images are stored as layers, and every layer is preserved forever. If you `COPY .env` in a Dockerfile, the secrets are in the image even if you later `RUN rm /app/.env` — the deleted state is in a later layer, but the file is still in the earlier layer where anyone with `docker history` or layer extraction can pull it out.

Secrets live in `.env` files on the host that runs the container, injected at runtime via `env_file:` in compose. The image stays generic and disposable; the host has the secrets.

Practical consequence: `.dockerignore` always includes `.env*`. CI build logs should never leak env vars. If a secret is ever in an image you pushed, treat it as compromised — rotate immediately.

2.4. CI builds, CD deploys

Two distinct phases, two distinct workflows.

CI (Continuous Integration) builds and pushes the image to GHCR. Its job is "is this code shippable?" If it produces a valid image and the tests pass, the answer is yes.

CD (Continuous Delivery / Deployment) takes a built image and makes it run on the VPS. Its job is "make prod match what's in the registry."

Splitting these makes failures debuggable. If CI fails, the bug is in the code or the Dockerfile. If CD fails, the bug is in the deployment plumbing (SSH key, compose file, network). You always know which layer to investigate.

Practical consequence: the build artifact (the image) is the boundary between the two phases. CD doesn't checkout your code, doesn't rebuild — it pulls the already-built image and runs it.

2.5. Separation of identity

- You SSH in as a sudo-capable admin user.
- CI SSHes in as a sandboxed `deploy` user.
- Apps connect to Postgres as per-app database users.

Established in `vps-from-zero`. The reason this matters for dockerized deployments: if a CI deploy key leaks, the attacker can mess with containers but can't `sudo`, can't read other apps' `.env` files (unless they're chained through a separate exploit), and can't access any database except via the running app.

3. The Dockerfile: the artifact that ties dev to prod

This is the single most important file in the whole pipeline. It defines what gets built, what gets shipped, and what runs in every environment.

The multi-stage approach

```
# syntax=docker/dockerfile:1.7

# — Base stage: shared dependencies —————
FROM python:3.12-slim AS base

# Install uv (fast Python package manager)
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

ENV UV_LINK_MODE=copy \
    UV_COMPILE_BYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

# Install runtime dependencies first (better layer caching)
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-install-project --no-dev

# — Dev stage: adds dev tools —————
FROM base AS dev

RUN apt-get update && apt-get install -y --no-install-recommends \
    git \
    curl \
    openssh-client \
    postgresql-client \
    && rm -rf /var/lib/apt/lists/*

# Install dev dependencies
RUN uv sync --frozen --no-install-project

# Don't COPY source – dev container mounts it as a volume
CMD ["bash"]

# — Runtime stage: minimal image for production —————
FROM base AS runtime

# Copy source code into the image
COPY src ./src
COPY alembic alembic
COPY alembic.ini ./

# Install the project itself
RUN uv sync --frozen --no-dev

# Create a non-root user
RUN useradd --create-home --shell /bin/bash app
USER app

CMD ["uv", "run", "python", "-m", "app.main"]
```

Why multi-stage at all?

Two stages have completely different goals:

- **Dev:** maximum convenience. Git, debuggers, shell tools, dev dependencies (pytest, ruff, mypy). You're going to be inside it interactively for hours.
- **Runtime:** minimum surface area. No debugger, no shell tools, no dev deps. Smaller image, fewer CVEs, fewer things for an attacker to use if they get a shell inside the container.

Multi-stage lets both share the parts that are the same — Python, uv, the runtime dependencies — without duplicating either the build commands or the resulting layers.

Concretely, the runtime image at the end of this Dockerfile is maybe 200MB. A naive single-stage image with both prod and dev deps and all the apt tools would be 700-900MB.

Walking through it, line by line

`# syntax=docker/dockerfile:1.7` Activates the modern Dockerfile syntax features (BuildKit). Without this, you'd be on a legacy parser that misses things like `RUN --mount=type=cache`. Pin a specific version so a Docker upgrade doesn't silently change your build.

`FROM python:3.12-slim AS base` Start from the official Python image, slim variant (no extra apt cruft). `AS base` names this stage so later stages can `FROM base`.

The `python:3.12-slim` tag is not immutable — `python:3.12-slim` today might be different than the same tag in six months. For maximum reproducibility, you could pin to a digest: `python:3.12-slim@sha256:abc...`. In practice, the slim images are very stable and the tradeoff isn't worth it for most apps.

`COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv` Cross-image copy: take the `uv` binary from the official uv image and put it in our image. This is faster than `pip install uv` and avoids a Python-bootstrapping step. It's also a great trick to remember for any tool that publishes its own Docker image.

`ENV UV_LINK_MODE=copy ...` Set environment variables for the rest of the build and for the container's runtime:

- `UV_LINK_MODE=copy` — uv normally uses hardlinks when installing packages (faster), but hardlinks don't survive Docker layer boundaries. `copy` mode is slower at install time but produces correct layers.
- `UV_COMPILE_BYTECODE=1` — compile `.pyc` files at install time, so they're ready when the container starts (saves ~100ms on cold start). Trades disk for startup speed.
- `PYTHONUNBUFFERED=1` — make stdout/stderr flush immediately. Without this, `print()` output can sit in a buffer for ages, which is awful for container logs.
- `PYTHONDONTWRITEBYTECODE=1` — at runtime, don't write fresh `.pyc` files (we already did that at build). Keeps the filesystem read-only-ish.

`WORKDIR /app` All subsequent paths are relative to `/app`. Also creates the directory if missing.

`COPY pyproject.toml uv.lock ./` and `RUN uv sync --frozen --no-install-project --no-dev` This is the **layer-caching trick** that makes builds fast.

Docker caches each `RUN` step's resulting filesystem. If the inputs haven't changed (file contents, command), Docker reuses the cached layer. By copying only the dependency manifest and running install before copying the actual source code, we ensure that:

- Code changes (which happen many times per day) don't invalidate the dep-install layer.

- Dep changes (which happen occasionally) do invalidate it — correctly.

`--frozen` means "fail if the lockfile is out of sync with the manifest" (you should always commit `uv.lock` alongside `pyproject.toml`). `--no-install-project` means "install dependencies but not the project itself yet — that comes after we copy the source." `--no-dev` excludes dev-only dependencies (pytest, mypy, etc.).

FROM base AS dev A second stage that inherits everything in `base`. Subsequent commands add on top.

RUN apt-get update && apt-get install -y --no-install-recommends git curl openssh-client postgresql-client && rm -rf /var/lib/apt/lists/* Install OS packages needed during development but not in production:

- `git` — for committing from inside the container (less needed if you commit from outside).
- `curl` — for testing HTTP endpoints.
- `openssh-client` — for `git push` (needs to talk to GitHub over SSH).
- `postgresql-client` — gives you `psql` inside the container for ad-hoc DB queries.

The `--no-install-recommends` flag prevents apt from also installing "recommended" packages (which are often graphical or documentation packages totally useless in a container). The `rm -rf /var/lib/apt/lists/*` at the end deletes the apt package index, which adds 30-40MB to the layer if left in place.

These three things — single-line install, `--no-install-recommends`, and cleanup — should be muscle memory for every Dockerfile.

RUN uv sync --frozen --no-install-project Install all dependencies including dev ones. Same caching logic as the runtime install — but in a separate layer in the `dev` stage, so it doesn't pollute the runtime image.

CMD ["bash"] The dev container's default command is a bash shell. VS Code's Dev Container extension will override this anyway (see Step 5), but having `bash` here makes the image usable for ad-hoc `docker run -it` testing.

FROM base AS runtime Third stage, inherits from `base`, ignores the `dev` stage entirely. Production builds target this stage with `--target=runtime` in the CI workflow.

COPY src ./src etc. Copy the actual application source into the image. Notice this is after the dependency install — so changing a file in `src/` only invalidates the cache from this COPY downward.

RUN uv sync --frozen --no-dev Install the project itself (now that the source is here). Still `--no-dev` to keep dev deps out of production.

RUN useradd --create-home --shell /bin/bash app Create a non-root user for the application to run as.

USER app Subsequent commands and the eventual **CMD** run as `app`, not `root`. This is a **defense-in-depth** measure: if an attacker exploits a bug in your application code, they're constrained to the `app` user's filesystem permissions, not root inside the container. Combined with the container's own isolation (which is real but not absolute), this adds another layer.

CMD ["uv", "run", "python", "-m", "app.main"] The default command when the container runs. `uv run` activates the venv uv built and then runs the command. `python -m app.main` runs the module as a script.

What this Dockerfile produces

After building with `--target=runtime`, you have a ~200MB image whose filesystem is:

```
/usr/local/bin/python3.12
/usr/local/bin/uv
/app/
  pyproject.toml
  uv.lock
  src/...
  alembic/...
  alembic.ini
  .venv/...
```

No git, no curl, no debuggers, no test framework, no shell utilities. The smallest, safest, most reproducible thing that can run your code.

4. The repo, end to end

```
apm-class-bot/
├── .devcontainer/
│   └── devcontainer.json      # VS Code dev container config
├── .github/
│   └── workflows/
│       ├── build.yml         # CI: build image, push to GHCR
│       └── deploy.yml        # CD: SSH to VPS, restart
├── alembic/                  # database migrations
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
├── alembic.ini
├── src/
│   └── app/
│       ├── __init__.py
│       └── main.py
├── tests/
├── .dockerignore             # files to exclude from build context
├── .env.example              # committed template of required env vars
├── .env                      # real local values, gitignored
├── .gitignore
├── docker-compose.yml        # local dev only
├── Dockerfile                # the ONE Dockerfile, multi-stage
├── pyproject.toml
├── README.md
└── uv.lock
```

The full point of this layout: **one Dockerfile drives every environment**. The dev container uses the `dev` stage. The local docker-compose uses the `runtime` stage by default. CI builds the `runtime` stage and pushes to GHCR. The VPS pulls that same image.

Why `.dockerignore` is one of the most important files

Without it, `COPY . .` (or even `COPY src/ ./src/` followed by another `COPY`) pulls in everything: `.env`, `.git`, `__pycache__`, `node_modules` from a stray `npm install`, your editor's swap files, build artifacts. Three problems:

1. **Secrets leak into image layers.** `.env` should never end up inside the image (Principle 2.3). If it does, every push to GHCR puts your secrets in a publicly-pullable place (for public repos) or in the layer cache (for private).
2. **Image size balloons.** A bloated `.git` directory is many MB of objects that contribute nothing at runtime. `__pycache__` is duplicate compiled bytecode (we already did this once via uv).
3. **Cache invalidation fires constantly.** Docker's layer cache invalidates when any file in the `COPY` source has changed. If `.git` is in there, every commit changes `.git`, invalidating layers that have nothing to do with `.git`.

A reasonable `.dockerignore`:

```
# secrets
.env
.env.*

# version control
.git
.gitignore

# python
__pycache__
*.pyc
.venv
.pytest_cache
.mypy_cache
.ruff_cache

# editor
.vscode
.idea
*.swp

# docs / artifacts not needed at runtime
docs/
*.md
!README.md
```

Why `.env.example` matters

Committed, with no real secrets, just placeholders:

```
DISCORD_TOKEN=replace-me-with-dev-bot-token
DATABASE_URL=postgresql://botuser:dev@postgres:5432/botdb
LOG_LEVEL=DEBUG
```

This is **the contract**. Any developer (including future-you in six months) knows exactly which env vars the app reads. They can copy `.env.example` to `.env` and fill in real values. Without this, every onboarding starts with grepping the code for `os.getenv(...)` calls.

The local `docker-compose.yml`

This file runs only in local dev. Production uses a different compose file that lives on the VPS (we'll get to it).

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: botuser
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: botdb
    ports:
      - "5432:5432"
    volumes:
      - postgres_dev_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U botuser -d botdb"]
      interval: 5s
      timeout: 3s
      retries: 5

  bot:
    build:
      context: .
      target: runtime
    env_file: .env
    depends_on:
      postgres:
        condition: service_healthy
    develop:
      watch:
        - action: sync
          path: ./src
          target: /app/src
        - action: rebuild
          path: pyproject.toml

  migrate:
    build:
      context: .
      target: runtime
    profiles: ["migrate"]
    env_file: .env
    depends_on:
      postgres:
        condition: service_healthy
    command: uv run alembic upgrade head

volumes:
  postgres_dev_data:
```

Why each piece is the way it is:

postgres service - Local dev gets its own Postgres in a container. Why not use the VPS one? Because we want to develop offline, run destructive tests against the database without affecting prod, and get a fresh DB whenever we want with `docker compose down -v`. - `ports: ["5432:5432"]` exposes Postgres on the host's localhost. This is opposite of the VPS rule (no

exposed ports), and that's correct — you want DBeaver/pgAdmin/psql on your laptop to be able to connect. The local DB is a toy with throwaway data; the prod DB is precious. - `healthcheck` is the same `pg_isready` pattern as the VPS setup. The bot won't start until Postgres is ready.

bot service - `build: .` and `target: runtime` build the same Dockerfile, same target as production. We're not running a dev-stage image here — we're running the production image, locally. This is the "one image, many environments" principle in action. - `env_file: .env` pulls env vars from your local (gitignored) `.env`. - `depends_on.postgres.condition: service_healthy` waits for the healthcheck to pass before starting the bot.

develop.watch This is the modern compose feature for **live development sync**. When a file under `./src` changes on your host, compose copies it into the running container at `/app/src`. No rebuild needed for code changes.

The rule `action: rebuild path: pyproject.toml` says: if the dependency manifest changes, rebuild the image (because deps changed and need to be reinstalled). This is the right behavior: code changes are cheap (sync), dep changes are expensive (rebuild) — and the file watch knows which is which.

Under the hood, `develop watch` uses `inotify` (Linux) or `FSEvents` (macOS) to watch the host filesystem. When a file change is detected, compose uses `docker cp` to copy the file into the container. The container doesn't restart; the file just appears in `/app/src`. Your app, if it has hot-reload (uvicorn, flask, watchexec), picks up the change.

migrate service A second instance of the same image, with `profiles: ["migrate"]` so it doesn't start by default. The `command:` override runs Alembic instead of the bot. We'll come back to this in §10 — it's the heart of safe deployments.

5. Local development with VS Code Dev Containers

The promise: your editor, debugger, language server, and Python runtime are all inside the same Linux container that ships to prod. No "I have the wrong Python version locally." No "this works in CI but not on my laptop." No juggling pyenv/asdf/mise.

The `.devcontainer/devcontainer.json`

```
{
  "name": "apm-class-bot dev",
  "dockerComposeFile": [ "../docker-compose.yml" ],
  "service": "bot",
  "workspaceFolder": "/app",
  "shutdownAction": "stopCompose",

  "build": {
    "target": "dev"
  },

  "overrideCommand": true,
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-python.python",
        "ms-python.vscode-pylance",
        "charliermarsh.ruff",
        "tamasfe.even-better-toml",
        "ms-azuretools.vscode-docker",
        "mtxr.sqltools",
        "mtxr.sqltools-driver-pg"
      ],
      "settings": {
        "python.defaultInterpreterPath": "/app/.venv/bin/python",
        "python.terminal.activateEnvironment": false,
        "editor.formatOnSave": true,
        "[python]": {
          "editor.defaultFormatter": "charliermarsh.ruff"
        }
      }
    }
  },

  "forwardPorts": [5432],
  "remoteUser": "root"
}
```

What every field does, mechanically

`dockerComposeFile + service` Instead of using a standalone Dockerfile, the dev container is part of a compose project. VS Code starts the whole compose project (bringing up `postgres` and `bot`), then attaches to the `bot` service.

This is the magic of using compose — your dev container gets the Postgres service "for free," with the same networking and same hostnames as production. You can `psql -h postgres` from inside the dev container and it just works.

`workspaceFolder: "/app"` Where your repo gets mounted inside the container. VS Code opens this directory when it attaches. The compose file already mounts the repo here via `develop.watch` syncing, so file changes are bidirectional (kind of — see below).

`shutdownAction: "stopCompose"` When you close VS Code, the dev container's compose project gets stopped. Without this, the containers would linger forever, eating memory.

build.target: "dev" Build the `dev` stage of the Dockerfile, not the default. VS Code passes this to compose's build process. So your dev container is the dev stage; your production image is the runtime stage; same Dockerfile, different targets.

overrideCommand: true The Dockerfile's `CMD ["bash"]` would exit immediately when run non-interactively (bash with no input → EOF → exit). VS Code's dev container needs a long-running process to attach to. With `overrideCommand: true`, VS Code replaces the `CMD` with its own idle process that keeps the container alive indefinitely. You then use the VS Code terminal to launch whatever you actually want to run.

customizations.vscode.extensions Extensions to install inside the container. Your local VS Code installation's extensions don't automatically transfer — each dev container has its own set, scoped to that container. List the ones you want here, and they auto-install the first time the container starts.

This is one of the most underrated features of the whole system: every developer who opens this repo's dev container gets the same set of extensions, configured the same way. No "wait, do you have the Python extension installed?"

customizations.vscode.settings Same idea: VS Code settings scoped to the container.

`python.defaultInterpreterPath: /app/.venv/bin/python` points VS Code at the venv uv created during the build, so type checking and IntelliSense use the exact dependencies the container has.

forwardPorts: [5432] Surface ports from the container to your host. Even though the local compose file already exposes 5432 on the host, this makes VS Code aware of it and shows it in the Ports panel.

remoteUser: "root" Which user VS Code runs as inside the container during dev. Root is fine for development (you need to `apt install` things occasionally). Production runs as `app` (set in the Dockerfile's `USER app` directive) — that doesn't apply here because we're using the dev stage, which doesn't have that `USER` directive.

Opening the project in a dev container

Concrete sequence:

1. Open VS Code on Windows.
2. `Ctrl-Shift-P` → "Remote-WSL: Open Folder in WSL" → navigate to `~/dev/your-repo`.
3. Bottom-left of VS Code now says "WSL: Ubuntu".
4. `Ctrl-Shift-P` → "Dev Containers: Reopen in Container".
5. First time: VS Code runs `docker compose up`, builds the `dev` stage of your image (1-3 minutes), attaches.
6. When ready, bottom-left says "Dev Container: apm-class-bot dev".
7. Open the integrated terminal — you're inside the container.

What "Reopen in Container" actually does, mechanically

1. VS Code reads `.devcontainer/devcontainer.json`.
2. Resolves `dockerComposeFile` and `service` — finds your compose file and the `bot` service.
3. Adds an extra compose override that includes `build.target: "dev"` and `command: "sleep infinity"` (or similar) to keep the container alive.
4. Runs `docker compose up -d` to bring up the whole project (postgres + bot).
5. Once `bot` is running, runs `docker exec -it <bot-container> /bin/sh` to attach a shell.

6. Inside the container, downloads and installs the VS Code "server" component — a remote agent that proxies LSP/extensions/terminal between the container and your local VS Code window.
7. Installs the listed extensions inside the container (only on first run; cached afterward).
8. Mounts your repo into `/app` (already done by the compose file).
9. Forwards the listed ports.
10. Reopens the window connected to the remote agent.

When you close the window: VS Code disconnects, runs `docker compose down` (per `shutdownAction: stopCompose`).

When you reopen the container later: most of this is cached. Subsequent opens are 5-10 seconds.

What's running where

Inside the dev container: - VS Code Server (the remote agent) - Your shell sessions (via integrated terminal) - The Python interpreter (when you run/debug code) - The language server (Pylance) - The formatter (Ruff)

On your host (laptop): - VS Code UI - Files in `~/dev/your-repo/` (bidirectionally synced with `/app` inside the container)

In a sister container (also part of the compose project): - Postgres

You can `Ctrl-Shift-P` → "Dev Containers: Show Container Log" to see what's happening at any time.

SSH agent forwarding — what makes `git push` work inside the container

You haven't installed your SSH key inside the container. You haven't copied it in. So how does `git push` to GitHub work from inside the dev container?

VS Code automatically forwards your WSL `ssh-agent` socket into the container.

Mechanically: your WSL `ssh-agent` listens on a Unix socket (e.g. `/tmp/ssh-XXX/agent.NNN`). The agent holds your unlocked private key. VS Code bind-mounts this socket into the container at a path the container can use, and sets `SSH_AUTH_SOCK` in the container's environment to point at it.

When you run `ssh git@github.com` (or `git push`, which calls `ssh` under the hood), `ssh` connects to that socket, asks the agent for an auth token, and the agent signs the challenge — all without the key ever leaving your host.

If you ever wonder why `ssh-add -L` inside the container shows the same keys as it does outside: that's why.

What you get for free inside the container

- **Same Python version as production.** No drift possible.
- **Same OS packages.** `apt list --installed` matches what'll run on the VPS.
- **Pre-installed dependencies.** No `pip install` step on first open — the Dockerfile did it during build.
- **Hot reload.** Edit a file under `src/`, save it, the container sees the new content immediately (via `develop.watch.sync`).
- **Postgres at hostname `postgres`.** Try `psql -h postgres -U botuser -d botdb`.
- **SSH agent forwarded for git.**

- All listed VS Code extensions installed automatically.

Running, testing, debugging from inside

```
uv run python -m app.main      # run the bot
uv run pytest                  # tests
uv run ruff check               # lint
uv run ruff format              # format
uv run alembic upgrade head     # apply migrations
```

Same commands you'd run in CI. They produce the same results because they're running in the same environment.

Rebuilding when the Dockerfile changes

If you edit `Dockerfile` or `pyproject.toml`:

`Ctrl-Shift-P` → "Dev Containers: Rebuild Container". 30-90 seconds.

Smaller changes (pure code) don't need a rebuild — they're synced live via `develop.watch`.

6. Day-to-day development cycles

What an actual day looks like inside this setup.

Morning routine

```
# in WSL
cd ~/dev/your-repo
git pull --rebase
code .                    # opens VS Code with Remote-WSL
```

In VS Code: if the dev container isn't already running, `Ctrl-Shift-P` → "Dev Containers: Reopen in Container" (~10s cached).

Inside the dev container terminal:

```
uv run pytest              # green before you change anything?
```

If yes, you're set. If red, you have your first task: figure out what changed on main while you were away.

The feature loop (most common)

You want to add a new feature.

```
# branch off main
git checkout -b feature/leaderboard

# edit src/app/commands/leaderboard.py
# write a test in tests/test_leaderboard.py

uv run pytest tests/test_leaderboard.py -v
# iterate until green

uv run ruff check
uv run ruff format

git add src/app/commands/leaderboard.py tests/test_leaderboard.py
git commit -m "add leaderboard command"
git push -u origin feature/leaderboard
```

Open a PR (`gh pr create --fill`) and either:

- **Merge directly** if you're solo and confident: `gh pr merge --auto --squash`.
- **Self-review** otherwise (worthwhile even solo — your future self benefits from the PR description as a changelog).

When the PR merges to main:

1. GHA `build.yml` builds the runtime image and pushes to GHCR (~90s).
2. GHA `deploy.yml` SSHes into the VPS and restarts the bot (~10s).
3. Total: ~2 minutes from merge to live.

Verifying the deploy worked

From your laptop (outside the dev container, in WSL):

```
ssh myvps "docker compose -f ~/apm-bot/docker-compose.yml logs --tail=20 bot"
```

Or for a live tail:

```
ssh myvps "docker compose -f ~/apm-bot/docker-compose.yml logs -f bot"
```

You should see the bot reconnecting to Discord with the new code.

Adding a new dependency

You want to add `httpx`.

```
# inside the dev container
uv add httpx
# this updates pyproject.toml and uv.lock
```

VS Code's dev container detects the `pyproject.toml` change (via the `develop.watch` rule with `action: rebuild`) and rebuilds the container. Or you can force it: `Ctrl-Shift-P` → "Dev Containers: Rebuild Container".

After rebuild, the new dep is installed. Commit both files:


```
git add pyproject.toml uv.lock
git commit -m "add httpx"
git push
```

CI rebuilds the prod image with the new dep. Prod gets the dep on next deploy.

Never commit `pyproject.toml` without `uv.lock`. The lockfile guarantees identical installs across environments. Same for `package-lock.json` / `poetry.lock` / `Cargo.lock` / etc.

Fixing a bug in production

Users report the bot is throwing errors on `!status`.

Look at prod logs.

```
ssh myvps "docker compose -f ~/apm-bot/docker-compose.yml logs --tail=200 bot | grep -A 10 ERROR"
```

You see a traceback. Note the file and line.

Reproduce locally.

In the dev container, write a failing test that reproduces the bug:

```
def test_status_with_no_active_users():
    # the exact condition from the prod traceback
    ...
```

`uv run pytest tests/test_status.py -v`. Red. Good — you've now captured the bug as a regression test.

Fix.

Edit, get the test green.

Ship.

```
git checkout -b fix/status-no-users
git add -A
git commit -m "fix: handle empty user list in !status"
git push -u origin fix/status-no-users
gh pr create --fill && gh pr merge --auto --squash
```

~2 minutes later, prod is fixed. Verify with prod logs again.

Debugging in production (when local can't repro)

Sometimes the bug only happens with prod data shapes. Options, in order of preference:

Easy: snapshot prod, restore locally.

```
# on the VPS
ssh myvps
docker exec infra-postgres-1 pg_dump -U botuser -d botdb > /tmp/snap.sql

# locally (in WSL)
scp myvps:/tmp/snap.sql ./
docker compose down -v
docker compose up -d postgres
sleep 5
docker compose exec -T postgres psql -U botuser -d botdb < snap.sql
# now run the bot locally against prod-shaped data
```

Delete `snap.sql` when you're done; treat snapshots as sensitive.

Harder: shell into the prod container.

```
ssh myvps
cd ~/apm-bot
docker compose exec bot bash
# inside the container, run python REPL or one-off commands
```

Don't make a habit of this. It's tempting to "just edit one line" in prod, which leads to drift between the prod image and the source repo. If you find yourself needing more than read-only inspection, write a real fix and ship through CI.

Best: VS Code "Attach to Running Container" via Remote-SSH.

1. Install Remote-SSH extension in VS Code.
2. `Ctrl-Shift-P` → "Remote-SSH: Connect to Host" → `myvps`.
3. Inside that VS Code session: `Ctrl-Shift-P` → "Dev Containers: Attach to Running Container" → pick `apm-bot-bot-1`.
4. You now have a full VS Code IDE attached to the prod container.

Wild and powerful; use sparingly. Perfect for "I need to read prod files with syntax highlighting and run an interactive Python session." Wrong for "I need to change a line in prod."

When to nuke vs when to hot-reload

Change	What to do
Edit a <code>.py</code> file	Save. Hot reload (via <code>develop.watch.sync</code>).
Add a Python dep (<code>uv add ...</code>)	Rebuild dev container.
Change Dockerfile	Rebuild dev container.
Drop or rename a DB column	Write + run migration via <code>alembic upgrade head</code> .
DB feels corrupted, want a fresh slate	<code>docker compose down -v && docker compose up -d</code> .
Dev container seems wedged	"Dev Containers: Rebuild Container Without Cache".
Compose file changed	<code>docker compose down && docker compose up -d</code> .
Want to test "fresh user" scenario	<code>docker compose down -v</code> then <code>alembic upgrade head</code> .

7. The registry: GHCR

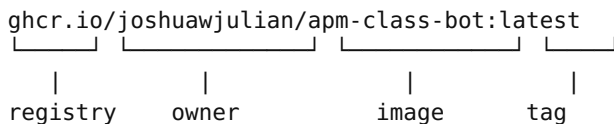
GHCR (GitHub Container Registry, `ghcr.io`) is GitHub's Docker image storage. Images live alongside your repo, visible in the "Packages" sidebar.

Why GHCR specifically

- **Free for personal repos** (public or private), with generous limits.
- **Auth is automatic in GitHub Actions** — the `GITHUB_TOKEN` that Actions hands you has permission to push to GHCR for that repo, no separate API key to set up.
- **Lives next to your code.** One UI, one place.

What the image address actually means

```
ghcr.io/joshuawjulia/apm-class-bot:latest
```



The diagram illustrates the components of the image address `ghcr.io/joshuawjulia/apm-class-bot:latest`. Brackets below the address group the parts into four categories: **registry** (`ghcr.io`), **owner** (`joshuawjulia`), **image** (`apm-class-bot`), and **tag** (`:latest`).

Everything before the `:` is the **image reference** — a globally-unique address. Different tags (`:latest`, `:sha-abc123`, `:v1.2.0`) are different versions of the image at that address.

Image visibility

By default, GHCR images created by GitHub Actions are **private**, linked to the repo, inheriting the repo's collaborators' access. For most personal projects, leave them private.

To make public: package page → Package settings → Change visibility → Public.

For a Discord bot or anything else that might have anything sensitive baked in, keep it private.

The personal access token (PAT) for pulling

The VPS needs to authenticate to GHCR to pull private images. Generate a **Personal Access Token (classic)** with `read:packages` scope only:

1. `github.com/settings/tokens` → "Generate new token (classic)".
2. Scope: only check `read:packages`.
3. Set expiration to 1 year (calendar reminder to renew).
4. Copy the token.

On the VPS, as `deploy`:

```
echo "<the-PAT>" | docker login ghcr.io -u <your-gh-username> --password-stdin
```

Docker stores this in `/home/deploy/.docker/config.json`. From now on, `docker pull ghcr.io/...` works for any image you own.

Why a PAT and not the same auto-token CI uses? The CI token only exists during a workflow run. The VPS pulls images on its own schedule (when you trigger a deploy), and at startup if a container needs to be restarted. It needs persistent credentials. The PAT is the simplest persistent credential.

Why `read:packages` and not broader scope? Principle of least privilege. This token can only pull images. If it leaks, the attacker can pull your private images — bad, but limited.

8. CI: building images with GitHub Actions

Create `.github/workflows/build.yml`:

```
name: Build and push image

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${github.actor}
          password: ${secrets.GITHUB_TOKEN}

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/${github.repository}
          tags: |
            type=raw,value=latest,enable={{is_default_branch}}
            type=sha,prefix=sha-
            type=ref,event=branch

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          target: runtime
          push: true
          tags: ${steps.meta.outputs.tags}
          labels: ${steps.meta.outputs.labels}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

What each block does, in depth

on: push: branches: [main] + workflow_dispatch Two triggers: every push to main, and manual triggering from the Actions tab UI. The manual trigger is invaluable for re-running a build that failed for a transient reason (registry hiccup, transient test failure) without needing a code change.

runs-on: ubuntu-latest GitHub provides clean Ubuntu VMs for each job. `ubuntu-latest` currently means Ubuntu 24.04. Pinning to `ubuntu-24.04` is more reproducible if you care about avoiding silent breakages when GitHub bumps the latest tag.

permissions: contents: read, packages: write

IMPORTANT. By default, the auto-generated `GITHUB_TOKEN` is read-only. `packages: write` is what lets it push to GHCR. Without it, you get a confusing "denied" error on push. `contents: read` is what lets the checkout step read your repo.

The token's permissions are scoped to the workflow run. After the workflow finishes, the token is invalidated. So even if a build script somehow exfiltrates the token, it's useless after the run.

docker/setup-buildx-action@v3 Enables **Buildx**, Docker's modern builder. The legacy builder (`docker build`) is still available but Buildx gives you:

- **Better caching** (`type=gha` and `type=registry` cache backends).
- **BuildKit features** — `RUN --mount=type=cache` for persistent build-time caches, `RUN --mount=type=secret` for build-time secrets that don't end up in layers.
- **Multi-platform builds** (linux/amd64 + linux/arm64 in one go).
- **Parallel stage execution** — independent stages of a multi-stage Dockerfile can build concurrently.

You don't have to use any of those features, but enabling Buildx now means they're available when you need them.

docker/login-action@v3 Authenticates the docker CLI on the runner to GHCR. The auth token is the temporary `GITHUB_TOKEN` (different from the PAT the VPS uses).

`username: ${{ github.actor }}` is whoever triggered the workflow (you, when you push). For organization-owned repos, this would be the org member who pushed.

docker/metadata-action@v5 Computes image tags. Looking at our config:

- `type=raw,value=latest,enable={{is_default_branch}}` — apply the `:latest` tag, but only when running on the default branch (main). Pushes from feature branches don't move `:latest`.
- `type=sha,prefix=sha-` — apply the `:sha-<commit-sha>` tag (e.g. `:sha-abc123def`). This is the **immutable rollback handle**.
- `type=ref,event=branch` — for non-main pushes, apply the branch name as a tag (e.g. `:my-feature-branch`).

So on a main push you get `:latest` and `:sha-abc123` simultaneously. On a feature branch push you get `:sha-abc123` and `:my-feature-branch` but NOT `:latest`.

docker/build-push-action@v5 Runs `docker buildx build` + `docker push` in one step.

- `context: .` — the build context is the repo root. This is what gets sent to the daemon (after `.dockerignore` filtering).
- `target: runtime` — build only the runtime stage. Buildx is smart enough to skip stages that aren't needed for the target.

- `push: true` — push after building. (Set to false in PR-builds-only setups.)
- `tags: ${ steps.meta.outputs.tags }` — tag the image with everything `metadata-action` computed.
- `labels: ${ steps.meta.outputs.labels }` — also apply OCI labels (build date, revision, source URL) that get embedded in the image.
- `cache-from: type=gha` and `cache-to: type=gha,mode=max` — use GitHub Actions cache as the layer cache backend.

How the cache works (and why `mode=max`)

Docker layer caching reuses layers when their inputs haven't changed. The cache backend is where those layers are stored between runs.

`type=gha` uses GitHub Actions' built-in cache (free, scoped to the repo). On the first run after a code change, only the layers that depend on the change get rebuilt; everything before that gets pulled from cache. Builds drop from 60s to 15s after the first build.

`mode=max` exports cache for all layers, not just the final image. This is necessary for multi-stage Dockerfiles — if you only export the final stage, the dev stage's expensive `apt-get install` layer never gets cached, and rebuilds of the dev stage (e.g. for Dev Container builds) are slow.

What CI produces

After `build.yml` runs to green:

- **In GHCR:** the image, tagged `:latest` and `:sha-<commit>`. Visible at `https://github.com/<you>/<repo>/pkgs/container/<repo>`.
- **In Actions logs:** the build output, useful for debugging.
- **As input to `deploy.yml`:** the `workflow_run` event that triggers CD.

CI's job is done. The image exists. Now CD runs.

9. CD: deploying to the VPS

Create `.github/workflows/deploy.yml`:

```

name: Deploy to VPS

on:
  workflow_run:
    workflows: ["Build and push image"]
    types: [completed]
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    if: ${ github.event.workflow_run.conclusion == 'success' }}

    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.VPS_HOST }}
          username: deploy
          key: ${ secrets.VPS_SSH_KEY }}
          script: |
            cd ~/apm-bot
            docker compose pull
            docker compose run --rm migrate
            docker compose up -d bot
            docker image prune -f

```

The `workflow_run` trigger explained

`workflow_run` fires when another workflow finishes. We're triggering `deploy.yml` on the completion of `build.yml`. The `if:` clause filters to successful completions only — a failed build doesn't deploy.

Why two separate workflows? Splitting build from deploy means:

- A failed build never produces a deploy.
- You can see the build/deploy status separately on the Actions tab.
- You can re-deploy without re-building (just trigger `deploy.yml` manually).
- The deploy can be disabled temporarily without breaking the build.

Alternative pattern: a single workflow with two jobs, where deploy `needs: build`. That works too — slightly faster, one less workflow file. Two workflows is slightly cleaner for log reading.

The SSH action

`appleboy/ssh-action@v1` SSHes into the VPS using the key in `VPS_SSH_KEY` (the GitHub secret you store the deploy key in) and runs the inline script.

Setting the secrets. In the repo: Settings → Secrets and variables → Actions → New repository secret:

Name	Value
<code>VPS_HOST</code>	Your VPS's IP or hostname
<code>VPS_SSH_KEY</code>	The private half of the deploy keypair (<code>cat ~/.ssh/vps-deploy</code>), full contents including <code>-----BEGIN/END OPENSSH PRIVATE KEY-----</code> lines

Security of the secrets: GitHub stores them encrypted at rest. They're only decrypted into the runner's environment during a workflow run, never logged or echoed back. If you ever need to rotate, just paste a new value in the same secret.

The script, line by line

```
cd ~/apm-bot
docker compose pull
docker compose run --rm migrate
docker compose up -d bot
docker image prune -f
```

`cd ~/apm-bot` — go to the app's stack directory on the VPS. Each app gets its own directory (Principle 2.2).

`docker compose pull` — pull the latest image (the one CI just pushed). Compose looks at the `image:` field in `docker-compose.yml` (which is `ghcr.io/.../apm-class-bot:latest`) and pulls it from the registry. If `:latest` now points at a new digest, the local copy is updated.

`docker compose run --rm migrate` — run the `migrate` service exactly once and remove the container afterward. The `migrate` service is defined in compose with `profiles: ["migrate"]`, so it doesn't start as part of `up -d`. The `run` command bypasses profiles.

If the migration fails, `run` exits with a non-zero code. The script (which has `set -e` implicit in shell or via `appleboy/ssh-action`'s default) stops here — `up -d bot` does NOT run. The old version of the bot keeps running. Failed migrations don't take down prod.

`docker compose up -d bot` — bring up (or update) the bot service. Compose compares the running container's config (image digest, env, etc.) with the desired state (from the compose file plus the just-pulled image). If they differ, the container is stopped and a new one started. If they're the same, nothing happens.

This is **idempotent and surgical**. Pulling a new image with the same tag changes the digest, which compose notices, which triggers a recreate. Pulling and finding no new image is a no-op.

`docker image prune -f` — clean up old image layers that are no longer referenced. Without this, the disk slowly fills with previous versions of the image. `-f` skips the confirmation prompt.

What `docker compose up -d` actually does, mechanically

This is worth understanding because it's the operational core of every deploy:

1. Parse `docker-compose.yml`.
2. For each service: a. Resolve the image (already pulled). b. Compute what config the container would have (image, env, mounts, command, ports, networks). c. Compare against any currently-running container with the same name. d. If config is identical → leave it alone. e. If config differs → stop the old container, start a new one with the new config.
3. Bring up any services that aren't running yet.
4. Return to caller.

So `up -d` after `pull` is the moment your new code starts running. The old container is stopped (graceful SIGTERM, then SIGKILL after 10s if it doesn't exit), and a new one starts from the new image.

Brief downtime during this transition (typically <5 seconds). For a Discord bot, this is fine — it reconnects automatically. For a web server, this is where you'd add a reverse proxy (Caddy) that buffers requests during the switch, or a more sophisticated zero-downtime deploy pattern.

What lives in the VPS-side compose file

`/home/deploy/apm-bot/docker-compose.yml` on the VPS (NOT in the repo):

```
networks:
  shared:
    external: true

services:
  bot:
    image: ghcr.io/joshuawjulian/apm-class-bot:latest
    restart: unless-stopped
    env_file: .env
    networks:
      - shared
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

  migrate:
    image: ghcr.io/joshuawjulian/apm-class-bot:latest
    profiles: ["migrate"]
    env_file: .env
    networks:
      - shared
    command: uv run alembic upgrade head
```

And `/home/deploy/apm-bot/.env`:

```
DISCORD_TOKEN=<production-token>
DATABASE_URL=postgresql://botuser:<prod-password>@postgres:5432/botdb
LOG_LEVEL=INFO
```

Differences from the local compose file:

- `image:` instead of `build:` — pull from registry, don't build.
- No `postgres` service — the bot connects to the shared infra Postgres over the shared network.
- `restart: unless-stopped` — auto-restart on crash or reboot.
- `logging` rotation — Docker's default JSON logs grow forever; this caps each container at 30MB (10MB × 3 files).
- `networks: [shared]` — join the shared network created in vps-from-zero.
- No `ports:` — the bot is outbound-only (Discord bots talk to Discord, not the other way). Web apps would expose ports here, then Caddy would reverse-proxy them.

IMPORTANT. The production `DISCORD_TOKEN` should be from a separate Discord application than your dev token. Bots can only be connected to Discord from one place at a time; if dev and prod use the same token, they'll fight.

10. Database migrations in the pipeline

This is the part of the pipeline where the abstractions stop helping. Schema changes are stateful, hard to roll back, and have the highest blast radius.

Why migrations are different from code

For most code changes, "deploy" means "replace the image." If the new image is broken, redeploy the old one. Reversible in seconds.

Schema changes are different:

- The database is **stateful**. The change happens once and stays.
- A bad migration can **corrupt data**. Rollback might be impossible.
- The new code and the new schema must arrive **together**, or one of them sees something it doesn't expect.

The discipline: treat migrations like a separate kind of deploy, with their own playbook.

Setting up Alembic

Inside the dev container, once:

```
uv add alembic sqlalchemy psycpg2-binary
uv run alembic init alembic
```

This creates:

- `alembic.ini` — config file.
- `alembic/` with `env.py`, `script.py.mako`, and a `versions/` folder for individual migrations.

Commit all of it.

Wire Alembic to your DATABASE_URL

In `alembic.ini`, find:

```
sqlalchemy.url = driver://user:pass@localhost/dbname
```

Comment it out. We'll set the URL from the environment instead.

In `alembic/env.py`, near the top:

```
import os
config.set_main_option("sqlalchemy.url", os.environ["DATABASE_URL"])
```

If you have SQLAlchemy models, also:

```
from app.db.models import Base
target_metadata = Base.metadata
```

Why drive Alembic from `DATABASE_URL`: so it uses the same connection string your app uses. One source of truth, no risk of pointing migrations at the wrong DB.

The migrate service in compose

Already in the compose files above. Repeating here:

```
migrate:
  image: ghcr.io/.../apm-class-bot:latest
  profiles: ["migrate"]
  env_file: .env
  networks: [shared]
  command: uv run alembic upgrade head
```

The `profiles:` mechanism, explained.

Services declared with a `profiles:` block are excluded from `docker compose up` unless either:

- You pass `--profile migrate` to `compose up`, or
- You target the service directly via `docker compose run` (which we do).

This is the cleanest way to define "a task that runs as part of this stack but isn't a long-running service." Migrations are exactly that — they run, do their work, exit. Without `profiles:`, every `docker compose up` would also start the migrate service, which would run migrations and exit, which is fine but produces noise.

`docker compose run --rm migrate` — start exactly one instance of the migrate service, with stdout/stderr attached, and remove the container afterward (`--rm`). If `alembic upgrade head` exits 0, the container exits 0. If migration fails, container exits non-zero, our deploy script bails.

Writing a migration

```
uv run alembic revision --autogenerate -m "add user table"
```

Creates `alembic/versions/abc123_add_user_table.py`:

```
"""add user table

Revision ID: abc123def456
Revises:
Create Date: 2026-05-24 12:34:56.789

"""
from alembic import op
import sqlalchemy as sa

revision = "abc123def456"
down_revision = None

def upgrade():
    op.create_table(
        "users",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("discord_id", sa.BigInteger(), nullable=False, unique=True),
        sa.Column("created_at", sa.DateTime(), server_default=sa.func.now()),
    )

def downgrade():
    op.drop_table("users")
```

IMPORTANT. READ THE GENERATED MIGRATION BEFORE COMMITTING. Autogenerate is a starting point, not a finished product. It sometimes:

- Misses column type changes (especially for custom types or enums).
- Generates operations in the wrong order.
- Produces an unsafe `downgrade()` (one that throws away data).

Testing locally against a fresh DB

Before pushing:

```
docker compose down -v          # nuke the local DB
docker compose up -d postgres
docker compose run --rm migrate
docker compose exec postgres psql -U botuser -d botdb -c "\dt"
uv run pytest
```

If migrations apply cleanly from empty to head and tests pass, it's safe to push.

Testing against a prod-shaped DB (for risky migrations)

For anything that touches existing data:

```
# on the VPS
docker exec infra-postgres-1 pg_dump -U botuser -d botdb > /tmp/snap.sql

# locally
scp myvps:/tmp/snap.sql ./
docker compose down -v
docker compose up -d postgres
sleep 5
docker compose exec -T postgres psql -U botuser -d botdb < snap.sql
docker compose run --rm migrate
```

This catches "works on a fresh DB but fails on real data shapes."

Wired into the deploy pipeline

Look at `deploy.yml` again:

```
script: |
  cd ~/apm-bot
  docker compose pull
  docker compose run --rm migrate      # <-- here
  docker compose up -d bot
  docker image prune -f
```

The order matters:

1. **Pull new image.** Now both the bot service and the migrate service refer to a newer image (they share the same `image:` field).
2. **Run migrations** using the new image. The migration script knows about new columns/tables because it's in the new image.

3. **Restart the bot** — only after migrations succeed. The bot now sees the new schema and uses the new code.

If migration fails (step 2), step 3 doesn't run. Bot keeps running on old image, old schema. You see the failure in the GitHub Actions UI, fix the migration, push.

The expand/contract pattern for destructive changes

Bad: a single migration that drops a column the app still reads.

Better: split into multiple deploys.

Step 1 (expand) — add the new column or table; code writes to old + new, reads from old:

```
def upgrade():
    op.add_column("users", sa.Column("username", sa.String(50)))
```

Step 2 (backfill) — populate the new column:

```
def upgrade():
    op.execute("UPDATE users SET username = name WHERE username IS NULL")
```

Step 3 (switch reads) — code reads `username` now, still writes to both.

Step 4 (contract) — drop the old column:

```
def upgrade():
    op.drop_column("users", "name")
```

Feels heavy for a personal bot. It is. But the moment you have data you care about and downtime you can't tolerate, this is the pattern.

Always back up before destructive migrations

```
# on the VPS, immediately before the risky deploy
docker exec infra-postgres-1 pg_dump -U botuser -d botdb \
    | gzip > ~/backups/pre-migration-$(date +%Y%m%d-%H%M%S).sql.gz
```

Restore:

```
gunzip < ~/backups/pre-migration-XXX.sql.gz \
    | docker exec -i infra-postgres-1 psql -U botuser -d botdb
```

Failed migrations: recovery playbook

Migration failed cleanly, error in CI logs:

- Bot is still on old image, old schema. No emergency.
- Check Alembic's notion of current revision: `bash ssh myvps "docker compose -f ~/apm-bot/docker-compose.yml run --rm migrate alembic current"`
- Fix the migration locally, test against prod snapshot, push.

Migration partially applied (e.g., ADD COLUMN succeeded but UPDATE failed):

- Schema is mid-state. Bot still on old image, but schema isn't where Alembic thinks it should be.

- Either push a corrective migration, or manually fix the DB: `bash docker exec -it infra-postgres-1 psql -U botuser -d botdb -- apply the operations Alembic didn't finish -- update Alembic's bookkeeping UPDATE alembic_version SET version_num = '<the-revision-id>';` Take a backup first.

Migration succeeded, new code is broken:

- Schema is forward-compatible (additive). Old code reading new schema is fine.
- Roll back the image to prior SHA tag. (See §12 Rollbacks.)

Destructive migration applied, you need to undo it:

- Restore from the pre-migration backup. This is why we always take one.

Alembic checklist before committing

- [] Did autogenerate pick up all the changes? Compare your model changes vs the migration ops.
- [] Is `downgrade()` usable, or does it throw away data?
- [] Any `op.drop_*` calls? Confirm they're intentional and you're doing expand/contract for anything risky.
- [] Column type changes? Postgres can't always cast in place — may need explicit `USING`.
- [] Indexes on big tables? Consider `postgresql_concurrently=True`.
- [] NOT NULL on existing columns? Need to backfill first.
- [] Foreign keys on populated tables? Re-validates every row; slow on big tables.

11. End-to-end: the full lifecycle of one code change

Let's walk through a single feature, end to end, watching every layer of the pipeline.

Scenario

You want to add a new Discord slash command: `/leaderboard`. It reads from a new `scores` table.

Step 1 — Branch and write the migration

Inside the dev container terminal:

```
git checkout -b feature/leaderboard
```

Add the SQLAlchemy model in `src/app/db/models.py`:

```
class Score(Base):
    __tablename__ = "scores"
    id = Column(Integer, primary_key=True)
    user_id = Column(BigInteger, ForeignKey("users.id"), nullable=False)
    value = Column(Integer, nullable=False)
    created_at = Column(DateTime, server_default=func.now())
```

Generate the migration:

```
uv run alembic revision --autogenerate -m "add scores table"
```

Read the generated file in `alembic/versions/`. Confirm it creates the table correctly.

Test the migration against a fresh DB:

```
docker compose down -v
docker compose up -d postgres
sleep 3
docker compose run --rm migrate
```

✓ Migration applies cleanly.

Step 2 — Write the command + tests

Add `src/app/commands/leaderboard.py` with the new slash command logic.

Add `tests/test_leaderboard.py`:

```
def test_leaderboard_with_no_scores(db_session, mock_interaction):
    result = await leaderboard_command(mock_interaction)
    assert "No scores yet" in result.content
```

Run tests:

```
uv run pytest tests/test_leaderboard.py -v
```

Iterate until green.

Step 3 — Lint + format

```
uv run ruff check
uv run ruff format
```

Step 4 — Commit and push

```
git add src/app/commands/leaderboard.py src/app/db/models.py tests/test_leaderboard.py
alembic/versions/*_add_scores_table.py
git commit -m "feat: add /leaderboard command with scores table"
git push -u origin feature/leaderboard
```

Step 5 — PR and merge

```
gh pr create --fill
gh pr merge --auto --squash
```

The PR squash-merges to main. **Now the pipeline starts.**

Step 6 — CI: build.yml fires

GitHub detects the push to main. `build.yml` starts running. You can watch it in the Actions tab.

What's happening on GitHub's runner:

1. `actions/checkout@v4` clones your repo into the runner's filesystem.
2. `docker/setup-buildx-action@v3` enables BuildKit.

3. `docker/login-action@v3` runs `docker login ghcr.io` with the auto-token.
4. `docker/metadata-action@v5` computes tags: `:latest` and `:sha-<the-merge-commit>`.
5. `docker/build-push-action@v5` runs `docker buildx build --target=runtime --tag ... --push .`. BuildKit pulls cached layers from the GHA cache where possible, builds new layers for changed parts (your code), and pushes the result to GHCR.

~90 seconds later, the build is green. In GHCR's UI, you can see the new image with `:latest` updated and a new `:sha-abc123` tag.

Step 7 — CD: `deploy.yml` fires

`workflow_run` fires on the successful completion of `build.yml`. `deploy.yml` starts.

What's happening on the runner:

1. `appleboy/ssh-action@v1` SSHes into your VPS as `deploy`.
2. The inline script runs: `cd ~/apm-bot docker compose pull # pull new :latest docker compose run --rm migrate # apply pending migrations docker compose up -d bot # restart bot with new image docker image prune -f # cleanup`

`docker compose pull` — the registry has a new `:latest` (different digest); Docker downloads the layers that aren't already cached on the VPS.

`docker compose run --rm migrate` — starts a one-shot container from the new image, runs `uv run alembic upgrade head`. Alembic connects to the shared Postgres (via `DATABASE_URL` in `.env`), finds the new migration file (`*_add_scores_table.py`), runs `CREATE TABLE scores ...`, updates the `alembic_version` table, exits 0. The container is removed.

`docker compose up -d bot` — Docker sees that the running `apm-bot-bot-1` container's image digest doesn't match the new `:latest` digest. Stops the old container (SIGTERM, then SIGKILL after 10s). Starts a new container with the new image.

Step 8 — The bot reconnects

Inside the new container, `uv run python -m app.main` runs. The bot:

1. Reads `DISCORD_TOKEN` from `/app/.env` (loaded from the `env_file` directive).
2. Reads `DATABASE_URL` and connects to Postgres at `postgres:5432`.
3. Connects to Discord's gateway WebSocket.
4. Registers slash commands (including the new `/leaderboard`).
5. Starts listening for events.

Step 9 — You verify

From your laptop:

```
ssh myvps "docker compose -f ~/apm-bot/docker-compose.yml logs --tail=20 bot"
```

You see:

```
2026-05-26 18:33:05 INFO Connected to Discord gateway
2026-05-26 18:33:06 INFO Registered 12 slash commands
2026-05-26 18:33:06 INFO Bot ready as MyBot#1234
```


Open Discord. Type `/leaderboard` in your test server. The new command appears in the autocomplete. Run it. ✓ Live.

What just happened, in time

Action	Time
You merge the PR	T+0
build.yml starts	T+5s
build.yml green	T+95s
deploy.yml fires	T+100s
<code>docker compose pull</code> finishes	T+115s
Migration runs	T+120s
<code>docker compose up -d</code>	T+125s
Bot reconnected to Discord	T+135s

From "merge button clicked" to "new feature live" in ~2 minutes. Without you SSHing in. Without you running a single command on the server. Without anyone needing to remember "did the migration run yet?"

That's the pipeline.

12. Rollbacks

A bad deploy can be reverted to the previous SHA-tagged image. This is why CI tags every build with `:sha-<commit>`, not just `:latest` — the SHA tags are **immutable**, so they always point at the same image bytes forever.

Image rollback

```
ssh myvps
cd ~/apm-bot
# edit docker-compose.yml - change `:latest` to `:sha-abc123def` (the prior known-good SHA)
docker compose pull
docker compose up -d bot
```

That's it. 30 seconds. The bot is back on the old version.

Why this works

`:latest` is mutable — it points at whatever was pushed most recently. `:sha-abc123def` is immutable — it points at the image built from that specific commit, forever.

When you pin compose to the SHA tag, `docker compose pull` fetches that specific image, even though `:latest` has moved past it. The next `up -d` notices the digest change and restarts the container with the rolled-back image.

To return to current after fixing the issue: change `:sha-...` back to `:latest` and pull/up again.

Schema rollback is harder

Image rollback doesn't undo migrations. If you rolled back the image because the migration was bad, you also need to fix the schema:

- **If the migration was additive** (added a column, added a table): old code can ignore the new column/table. No DB action needed.
- **If the migration was destructive** (dropped a column the old code reads): the old code will crash. You need to restore from the pre-migration backup, then redeploy.

This is why the expand/contract pattern matters. With expand/contract, the schema is always backward-compatible at every step, so image rollback always works.

13. Adding a second (or fifth) app

The marginal cost of app N+1 is ~10 minutes.

Checklist

For a new app called `dashboard`:

1. **Create the repo** with the same structure as your first one — Dockerfile, `.devcontainer/`, `.github/workflows/`, `docker-compose.yml`, `.env.example`. The fastest way is to make your first repo a GitHub template and create new repos from it.
2. **Add `VPS_HOST` and `VPS_SSH_KEY` secrets** to the new repo's settings.
3. **Create the database on the VPS:** `bash ssh myvps sudo -iu deploy docker compose -f ~/infra/docker-compose.yml exec postgres psql -U postgres sql CREATE USER dashuser WITH PASSWORD '<random>'; CREATE DATABASE dashdb OWNER dashuser;`
4. **Create the VPS directory:** `bash mkdir ~/dashboard # populate ~/dashboard/docker-compose.yml (point at ghcr.io/.../dashboard:latest) # populate ~/dashboard/.env (DATABASE_URL=postgresql://dashuser:~/dashdb)`
5. **Push to main.** CI builds, deploys, app comes up on the shared network and can reach Postgres at `postgres:5432`.

What's shared vs per-app

Shared across all apps	Per-app
The shape of the Dockerfile (multi-stage base/dev/runtime)	Dockerfile contents (deps, source)
The shape of <code>.devcontainer/devcontainer.json</code>	Dockerfile contents
The shape of GHA workflows	Image name in workflow
Directory structure on VPS (<code>~/<app-name>/</code>)	Stack directory and its contents
Shared <code>postgres</code> hostname in connection strings	Database name + user
The shared Docker network	—
Caddy reverse proxy (once you add it)	Caddyfile entry

Sharing common config across repos

If you start running 3+ apps, you'll feel the duplication. Options:

- **GitHub template repo.** Make one of your repos a template; new apps start as a fork-with-history.
- **Reusable workflows.** GitHub Actions supports `workflow_call`, letting multiple repos invoke a shared workflow file in an "infra" repo. Worthwhile at ~5 apps.
- **Scaffold script.** A `new-app.sh` that prompts for a name and stamps out the directory.

Don't optimize for this until you feel the pain. Premature DRY is harder to undo than copy-paste.

14. Reverse proxy with Caddy (HTTPS for web apps)

Discord bots don't need inbound HTTP. The moment you build something with a web UI (dashboard, REST API, etc.), you need:

- TLS certificates (auto-fetched via Let's Encrypt).
- Routing different domains to different containers.
- Port 80/443 exposed to the internet.

Caddy does all of this with the simplest config available.

Add Caddy to the infra stack

Edit `/home/deploy/infra/docker-compose.yml` (the one set up in vps-from-zero):

```
networks:
  shared:
    external: true

services:
  postgres:
    # ... (existing)

  caddy:
    image: caddy:2
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
      - "443:443/udp"      # for HTTP/3
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - shared

volumes:
  postgres_data:
  caddy_data:
  caddy_config:
```

The Caddyfile

`/home/deploy/infra/Caddyfile`:

```
dashboard.yourdomain.com {  
    reverse_proxy dashboard:3000  
}  
  
api.yourdomain.com {  
    reverse_proxy api-service:8080  
}
```

That's it. Two-line entries per app. Caddy:

- Solicits Let's Encrypt certificates for each hostname automatically (HTTP-01 challenge).
- Renews certs in the background ~30 days before expiry.
- Reverse-proxies requests to the named container on the shared network.

Why hostname `dashboard:3000` and not `localhost:3000`? Because Caddy is itself a container. `localhost` would mean Caddy's own loopback. The shared network's DNS resolves `dashboard` to the dashboard container's IP.

Open the firewall

```
sudo ufw allow 80/tcp  
sudo ufw allow 443/tcp
```

Point DNS

A records at your DNS provider:

```
dashboard.yourdomain.com → <vps-ip>  
api.yourdomain.com → <vps-ip>
```

Restart infra

```
cd ~/infra  
docker compose up -d  
docker compose logs -f caddy
```

Within ~10 seconds of first DNS resolution, Caddy fetches certs and serves HTTPS. `https://dashboard.yourdomain.com` works with a valid cert.

App-side: no exposed ports

Apps proxied by Caddy do **not** need `ports:` in their compose file. Caddy reaches them over the shared network. Skip `ports:` entirely — the container only listens on the shared network, never on the host's public IP. Defense in depth (same pattern as Postgres).

15. Troubleshooting

Layer-by-layer debugging strategy

When something breaks, work from "where you're at" backward through the pipeline:

Symptom	Layer to check
Code doesn't work in Dev Container	Dev container build, file sync
<code>git push</code> fails	SSH agent forwarding
<code>build.yml</code> red	Dockerfile, GHCR auth (permissions block)
<code>deploy.yml</code> red	SSH key in <code>VPS_SSH_KEY</code> , deploy user perms, compose file syntax
<code>docker compose pull</code> fails on VPS	GHCR auth on VPS (PAT)
Migration fails	Migration logic, schema state
Bot won't start after deploy	Container env vars, <code>DATABASE_URL</code> , network attachment
Bot starts but can't reach DB	<code>networks: [shared]</code> , hostname <code>postgres</code>

Common specific errors

"denied: installation not allowed to Write organization package" (in `build.yml`) You forgot `permissions: packages: write` in the workflow. Add it.

Permission denied (publickey) in deploy.yml The `VPS_SSH_KEY` secret is wrong, or the public half isn't in `/home/deploy/.ssh/authorized_keys` on the VPS.

Could not resolve hostname postgres from inside an app container The app container isn't on the `shared` network. Check the app's `docker-compose.yml`:

```
networks:
  shared:
    external: true

services:
  bot:
    # ...
    networks:
      - shared
```

Both blocks need to be there. The top-level `networks:` declares the network reference; the per-service `networks: - shared` attaches.

pull access denied for ghcr.io/... on the VPS Deploy user not logged into GHCR. As `deploy`:

```
echo "<PAT>" | docker login ghcr.io -u <username> --password-stdin
```

Bot starts but database queries hang Postgres health check wasn't satisfied before the bot started. The shared infra Postgres should have a `healthcheck`, and the app should declare `depends_on: postgres: condition: service_healthy` — but `depends_on` across compose projects (infra vs app stack) doesn't work the same way. In practice, by the time you're deploying apps, infra Postgres is already up. If it isn't: bring up infra first.

`ssh-add -L` **empty inside Dev Container** VS Code's SSH agent forwarding isn't working. Confirm `echo $SSH_AUTH_SOCK` in WSL (outside container) returns a path. Then quit VS Code completely (`taskkill /F /IM Code.exe`), reopen.

Image is huge (>500MB) - Missing `.dockerignore` (`.git`, `node_modules`, `__pycache__` are in the image). - Single-stage Dockerfile (dev tooling shipped in prod image). - `apt install` without `--no-install-recommends` and `rm -rf /var/lib/apt/lists/*`.

`docker history <image>` shows which layers are biggest.

Cannot start service postgres after a **Postgres major-version bump** You upgraded `postgres:16` → `:17` without a dump/restore. Recovery: revert to `:16`, dump, delete volume, bring up `:17`, restore. Don't use `:latest` for Postgres.

Migration says "Multiple heads" Two migrations branch from the same parent. Run `alembic heads`, then `alembic merge -m "merge" <rev1> <rev2>` to create a merge revision.

Autogenerate produces an empty migration `target_metadata` not set in `env.py`, or your models aren't imported there.

16. Alternatives considered

Image registry

GHCR (this guide) — free for personal, auth integrated with Actions. **Docker Hub** — fine for public; stingy on private. Higher trust ceiling on rate limits. **Self-hosted (Harbor, Distribution)** — only worth it if scale demands it.

CI/CD platform

GitHub Actions (this guide) — free for personal, integrates with GHCR. **GitLab CI** — equivalent if you're on GitLab. **Drone, Woodpecker, Jenkins** — self-hosted; more ops burden.

Dev environment

Dev Containers (this guide) — same image as production, zero drift. **Local venv** — fastest micro-iteration; risks toolchain drift. **Nix / devbox / mise** — reproducible without containers; steeper curve.

Migration tool

Alembic (this guide) — standard for Python/SQLAlchemy. **Yoyo migrations** — simpler, no ORM required. **Flyway / Liquibase** — Java ecosystem, SQL-first; overkill for personal. **Raw SQL files + a version table** — reinventing Alembic poorly.

PDF engine for migrations (...kidding)

Reverse proxy

Caddy (this guide) — simplest config, auto-HTTPS. **Traefik** — more powerful, Docker-aware, more YAML to write. **nginx** — battle-tested, but you write the cert renewal scripts.

Container engine

Docker Engine (this guide). **Podman** — daemonless, rootless by default. Compatible-ish with Docker but small gotchas. Worth considering for the security-conscious. **containerd directly** — too low-level for personal use.

17. Quick reference

Local development

```
# open the project in dev container
cd ~/dev/<repo>
code .
# then Ctrl-Shift-P → "Dev Containers: Reopen in Container"

# inside dev container
uv run python -m app.main          # run the app
uv run pytest                      # tests
uv run ruff check && uv run ruff format
uv add <package>                  # add dep, rebuild container
uv run alembic revision --autogenerate -m "..." # new migration
uv run alembic upgrade head        # apply migrations to local DB
```

Day-to-day shipping

```
git checkout -b feature/<name>
# code, test, lint
git push -u origin feature/<name>
gh pr create --fill && gh pr merge --auto --squash
# ~2 min later: live in prod
```

Watching a deploy

```
# from your laptop
ssh myvps "docker compose -f ~/<app>/docker-compose.yml logs -f --tail=50 <service>"
```

Operating the VPS as deploy

```
ssh myvps-deploy

# bring up an app
cd ~/<app> && docker compose up -d

# pull latest + restart (manual deploy)
cd ~/<app> && docker compose pull && docker compose up -d

# run migrations manually
cd ~/<app> && docker compose run --rm migrate

# logs
docker compose logs -f --tail=100 <service>

# shell into running container
docker compose exec <service> bash

# free disk
docker system prune -af
```

Rollback

```
ssh myvps
cd ~/<app>
# edit docker-compose.yml: change `:latest` to `:sha-<prior-commit>`
docker compose pull && docker compose up -d
```

The full deploy flow at a glance

[Diagram: see the online version on GitHub for the interactive flowchart.]

Total: ~90-120 seconds from push to live.

What lives where

Thing	Where	Notes
Source code	Your laptop + GitHub	Repo per app
Dev container	Your laptop (locally built)	Uses <code>dev</code> stage of Dockerfile
Production image	GHCR	Built by CI, pulled by VPS
App <code>docker-compose.yml</code> (local)	In repo	Includes Postgres service
App <code>docker-compose.yml</code> (prod)	VPS only, at <code>~/<app>/</code>	No Postgres; references shared one
App <code>.env</code> (local)	Your laptop, gitignored	Dev creds
App <code>.env</code> (prod)	VPS only, at <code>~/<app>/.env</code>	Prod creds
Personal SSH key	WSL <code>~/.ssh/id_ed25519</code>	Used by you, forwarded into containers
CI deploy key (private)	GitHub Secret <code>VPS_SSH_KEY</code>	Never on your laptop's agent
CI deploy key (public)	VPS <code>/home/deploy/.ssh/authorized_keys</code>	
GHCR pull token (PAT)	VPS <code>/home/deploy/.docker/config.json</code>	<code>read:packages</code> scope
Postgres data	VPS Docker volume <code>infra_postgres_data</code>	Backed up nightly
Per-app DB password	App's <code>.env</code> on VPS + password manager	One per app, never reused